Why Should You Integrate Git?

For a multi-script project like yours, using Git is like having a superpower for managing your code. It's much more than just a backup system. Think of it as a detailed lab notebook for your entire project that records every change you make.

Here are the key benefits for your specific workflow:

1. **Complete Version History:** Instead of saving files like `script_v1.R`, `script_v2_final.R`, `script_v2_final_final.R`, Git saves "snapshots" (called **commits**) of your entire project every time you tell it to. You can look back at any snapshot, see exactly what you changed, and even revert your entire project back to that state.
2. **Fearless Experimentation (Branching):** This is the most powerful feature. Imagine you want to try a completely different imputation method in your `2b_template_analysis_cps.R` script, but you're worried it might break things. With Git, you can create a separate "branch" to work on this new method.
    - Your main, working code remains safe and untouched on the `main` branch.
    - If the experiment works, you can merge the changes back into your main project.
    - If it fails, you can simply delete the experimental branch with no harm done.
3. **Mistake Recovery:** Did you delete a huge chunk of code and save the file, only to realize you needed it? Git has you covered. You can easily compare your current file to any previous version and restore the parts you need. It's the ultimate "undo" button.
4. **Meaningful Change Tracking:** Every time you save a snapshot (commit), you write a message describing what you did (e.g., "Updated ACS cleaning logic for POVERTY variable" or "Fixed bug in visualization function"). This makes it incredibly easy to see the history of your project and understand why certain changes were made months later.
5. **Collaboration and Backup (with GitHub/GitLab):** By connecting your local Git project to an online service like GitHub, you get two more massive benefits:
    - **Off-site Backup:** Your entire project history is safely stored online. If your computer is lost or damaged, your work is not gone.
    - **Collaboration:** It's the industry standard for working with others. If you ever bring on a collaborator, you can both work on the project simultaneously without overriding each other's files.

How to Integrate Git into Your RStudio Project

The best part is that RStudio has fantastic, built-in support for Git, so you rarely have to leave your development environment.

Prerequisites:

1. **Install Git:** You need to have Git installed on your computer. You can download it from git-scm.com.
2. **Create a GitHub Account:** Sign up for a free account at GitHub.com.

Step-by-Step Guide:

Step 1: Enable Git in Your RStudio Project

1. Open your RStudio project for the /Phase 2 directory.
2. Go to the menu bar and click Tools > Project Options.
3. In the options window, select the Git/SVN tab.
4. In the "Version control system" dropdown, select Git.
5. Click OK. RStudio will ask to confirm initializing a new Git repository and restarting R. Click Yes.

After RStudio restarts, you will see a new **Git tab** in the same pane as your Environment and History tabs. This is your mission control for version control.

Step 2: Make Your First Commit (Your Initial Snapshot)

1. Click on the Git tab. You will see a list of all the files in your project. These are all "unstaged" right now.
2. **Stage Files:** Check the boxes under the "Staged" column for all the files you want to include in your first snapshot (it's fine to select all of them). "Staging" means you're preparing them to be committed.
3. **Commit:** Click the Commit button. A new window will pop up.
4. **Write a Commit Message:** In the top-right pane, write a short, descriptive message. For the first one, something like "Initial commit of project templates" is perfect.
5. Click the Commit button in this window.

You have just saved your first snapshot! The file list in the Git tab will now be empty because there are no new changes to track.

Step 3: Connect to GitHub and Push Your Code

This step puts your project on GitHub for backup and sharing. The easiest way to do this is with the usethis package.

1. Install and load the usethis package in the R console:
   ```
   install.packages("usethis")
   library(usethis)
   ```

2. Run the use_github() command in the console:
   ```
   use_github()
   ```

3. This function will guide you through the process. It will:
   - Ask you to generate a "Personal Access Token" (PAT) on GitHub, which is like a secure password for your code. It will open the correct GitHub page for you.
   - Create a new repository on your GitHub account.
   - Connect your local project to that new repository.
   - Perform your first **push**, which sends your committed code up to GitHub.

Step 4: Your Daily Workflow

From now on, your workflow will be a simple, repeating cycle:

1. **Work:** Make changes to your R scripts—fix a bug, add a new plot, change a variable.
2. **Stage:** When you've reached a good stopping point, go to the Git tab, and check the boxes next to the files you've changed.
3. **Commit:** Click Commit, write a clear message describing your changes (e.g., "Updated ACS analysis to include cost burden metric"), and click the commit button.
4. **Push:** Click the green Push arrow in the Git tab to send your new commits to your GitHub repository for backup.

That's it! By following this cycle, you will have a complete, well-documented, and safely backed-up history of your entire project.